

Probabilistic Programming Languages: Snake Oil or Steroids?

Marc Light

Acknowledgements (so much is borrowed!)

- Illustrations are linked to their sources, when they come from others
- Model comes mostly from [Baio and Blangiardo](#) (Italian Soccer league modeling)
- [Pyro implementation](#) of modified version of B&B's model comes from Theo Rashid
- [Non-PPL implementation of Poisson Regression fit](#) (Martin Eastwood)
- Bayesian coin bias code comes from [Statistical Rethinking with PyTorch and Pyro](#)
- Basketball training data API comes from [Python Sports Analytics Made Simple \(Part 1\) — Creating a public sports API | by Robert Clark | Clarktech Sports](#)
- But any errors are likely from me

Outline of the talk

- Goals of the presentation and my motivations
- Simple Bayesian model and its implementations in PPL and non-PPL
 - 13 correct, 8 incorrect: how good is the prediction system?
 - A benefit of a Bayesian approach
 - Contrast implementations
- Desiderata for PPLs
- Generative Bayesian model for basketball game scores
 - Introduce the model
 - Contrast implementations
- Desiderata scorecard
- My brother's guaranteed way to win in Vegas
- Why the model worked at all, problems & next steps, why build generative models, why be a Bayesian

Goals of the talk

1. Indirectly introduce

- a. generative models,
- b. bayesian modeling,
- c. probabilistic programming languages,
- d. sportsbook drink tickets

} all of which are related

2. Tell the tale of my attempts to work with the above concepts

3. Help you decide where, if at all, the concepts fit into your work

Why did I commit my weekends and early mornings to exploring this stuff?

I had high hopes that PPLs would allow me to

- **Implement models quickly and effectively**
- **Focus on learning how to model better**

And I wouldn't have to

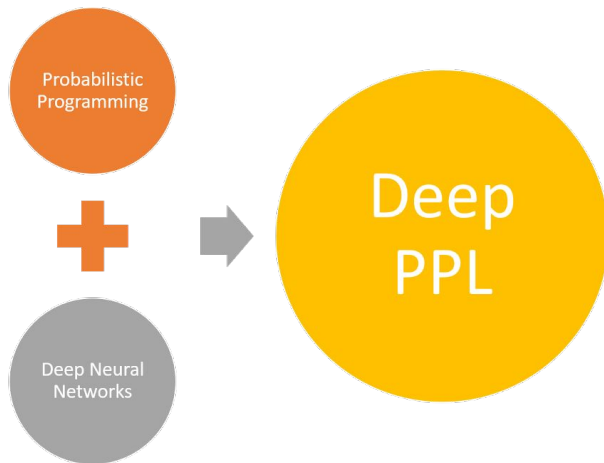
- **Spend 3+ months learning to use something not ready for prime time**
- **Fiddle with the nitty-gritty details of implementing inference algorithms such as MCMC, Inferential Bayes, etc.**

I thought others may have had similar thoughts as they were offered the snake oil/steroids for Stan, PyMC3, Pyro, Bayesian Modeling, etc.

Maybe	PPLs	are to	probabilistic modeling,
as	Matlab/R/Pandas	are to	linear algebra !!!

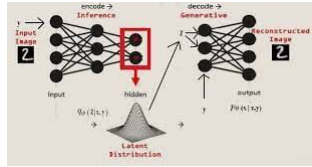
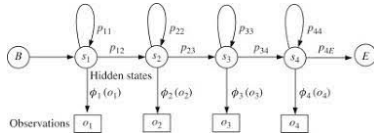
I am also intrigued by the combination of Bayesian Modeling and Deep Learning

“Pyro enables flexible and expressive deep probabilistic modeling, unifying the best of modern deep learning and Bayesian modeling.” (<https://pyro.ai/>)

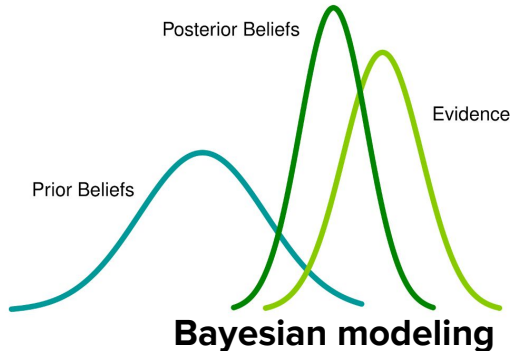


Speed intro

Images linked to sources



Generative models



$$\begin{aligned}\alpha &\sim \text{Normal}(0, 1) \\ \beta &\sim \text{Normal}(0, 1) \\ \sigma &\sim \text{Normal}_+(0, 1) \\ \hat{y}_j &= \alpha + \beta x_j \\ y_j &\sim \text{Normal}(\hat{y}_j, \sigma)\end{aligned}$$

```
m = @model x begin
    alpha ~ Normal()
    beta ~ Normal()
    sigma ~ HalfNormal()
    yhat = alpha .+ beta .* x
    n = length(x)
    y ~ For(n) do j
        Normal(yhat[j], sigma)
    end
end
```

Probabilistic Programming Language



Drink Tickets

Beat Vegas 13x, lost 8x: quit the day job?

θ : the bias of the system to winning

$P(\theta)$: the prior

$P(D | \theta)$: the likelihood

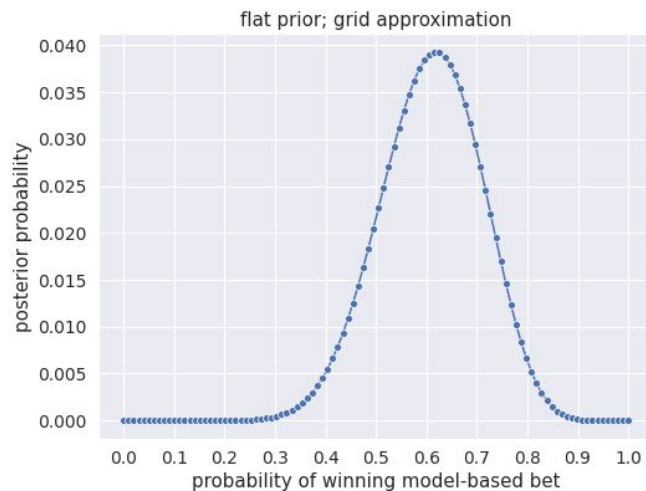
$P(\theta | D)$: the posterior

$P(\theta)$ = uniform(0 to 1)

$P(D | \theta)$ = $\binom{21}{13} \theta^{13} (1-\theta)^8$

$P(\theta | D)$ = $P(D | \theta) * P(\theta) / P(D)$

$P(D)$ = $\sum_{\theta} (P(D | \theta) * P(\theta))$

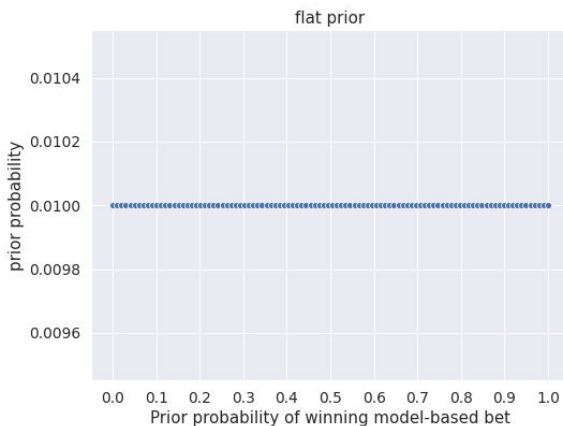


There is a 54% chance that the system is 60% right or more

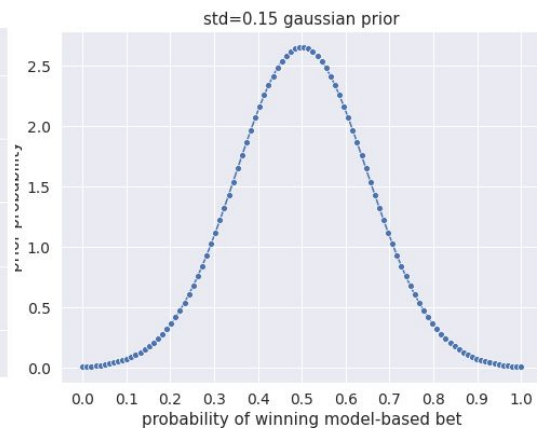
Playing with the prior (a benefit of Bayes)

The system is unlikely to be right 80% of the time.
We can encode this belief in the prior.

We choose a Gaussian, mean=0.5, stdDev = 0.15

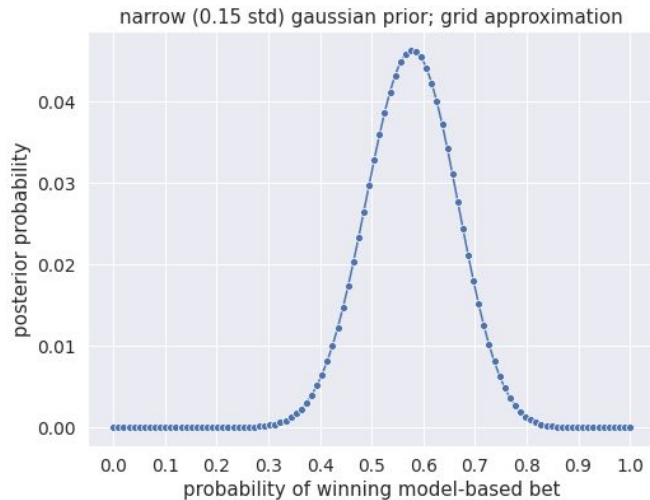


Old Prior



New Prior

New Posterior



Now there is a 39% chance that the system is 60% right or more

First look at some code

```
# define grid  
p_grid = torch.linspace(start=0, end=1, steps=100)  
  
# define prior  
prior = dist.Normal(0.5, 0.15).log_prob(p_grid).exp()  
  
# compute likelihood at each value in grid  
likelihood = dist.Binomial(total_count=21,  
                           probs=p_grid).log_prob(torch.tensor(13.)).exp()  
  
# compute product of likelihood and prior  
unstd_posterior = likelihood * prior  
  
# standardize the posterior, so it sums to 1  
posterior = unstd_posterior / unstd_posterior.sum()
```

Non-PPL code

Grid search is the inference method (p_grid). It is mixed in with the definition of the model.

*PPLs separate inference
model from the model*

```
def model2(w):  
    #prior  
    p = pyro.sample("p", dist.Normal(0.5, 0.1))  
  
    # binomial likelihood  
    pyro.sample("w", dist.Binomial(21, p), obs=w)  
  
globe_qa2 = MAP(model2).run(torch.tensor(13.))  
precis(globe_qa2)
```

PPL code

Marc's desiderata for PPLs

- Model specification should be
 - Separate from inference methods
 - In one place in the code
 - Easy to verify that code implements math
 - Code has direct mapping to math
 - Visualization tools are provided
- Inference and generation should have
 - Standardized interface making it easy to switch methods
 - Details hidden for things like numerical underflow
 - Tools for monitoring of progress, fit evaluation, & debugging tools
 - Efficient implementation!
- Documentation
 - Meaningful error messages
 - Great tutorial
 - Cookbook of example models+inference (like the galleries for Matplotlib, Seaborn, ggPlot, etc.)

A model for soccer scores (B&B & Rashid)

$$\sigma_{att} \sim \text{HalfStudentT}(3, 0, 2.5),$$

$$\sigma_{def} \sim \text{HalfStudentT}(3, 0, 2.5).$$

Generalize at
higher levels
in model

$$\alpha \sim \mathcal{N}(0, 1),$$

$$\text{attack}_t \sim \mathcal{N}(0, \sigma_{att}^2),$$

$$\text{defend}_t \sim \mathcal{N}(0, \sigma_{def}^2).$$

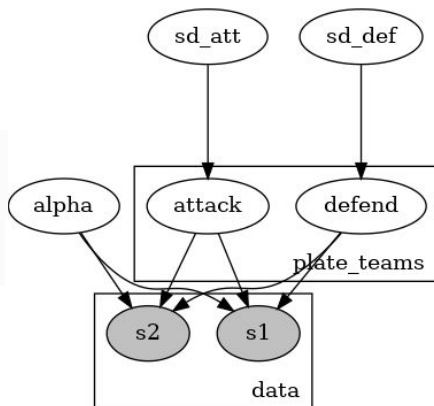
$$\log(\theta_{g1}) = \alpha + \text{attack}_{a(g)} - \text{defend}_{h(g)},$$

$$\log(\theta_{g2}) = \alpha + \text{attack}_{a(g)} - \text{defend}_{h(g)},$$

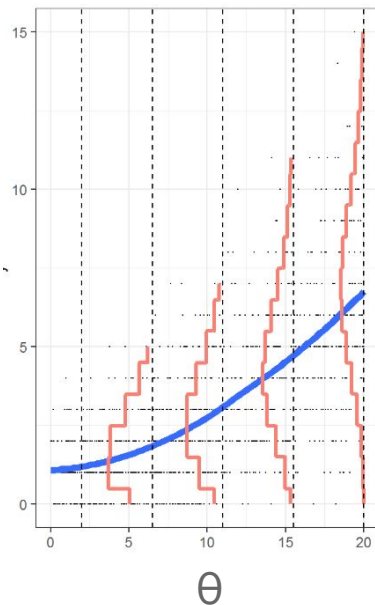
$$s_{gj} | \theta_{gj} \sim \text{Poisson}(\theta_{gj}).$$

Pyro generated this picture from
model function

```
alpha ~ Normal
sd_att ~ TransformedDistribution
sd_def ~ TransformedDistribution
attack ~ Normal
defend ~ Normal
s1 ~ Poisson
s2 ~ Poisson
```

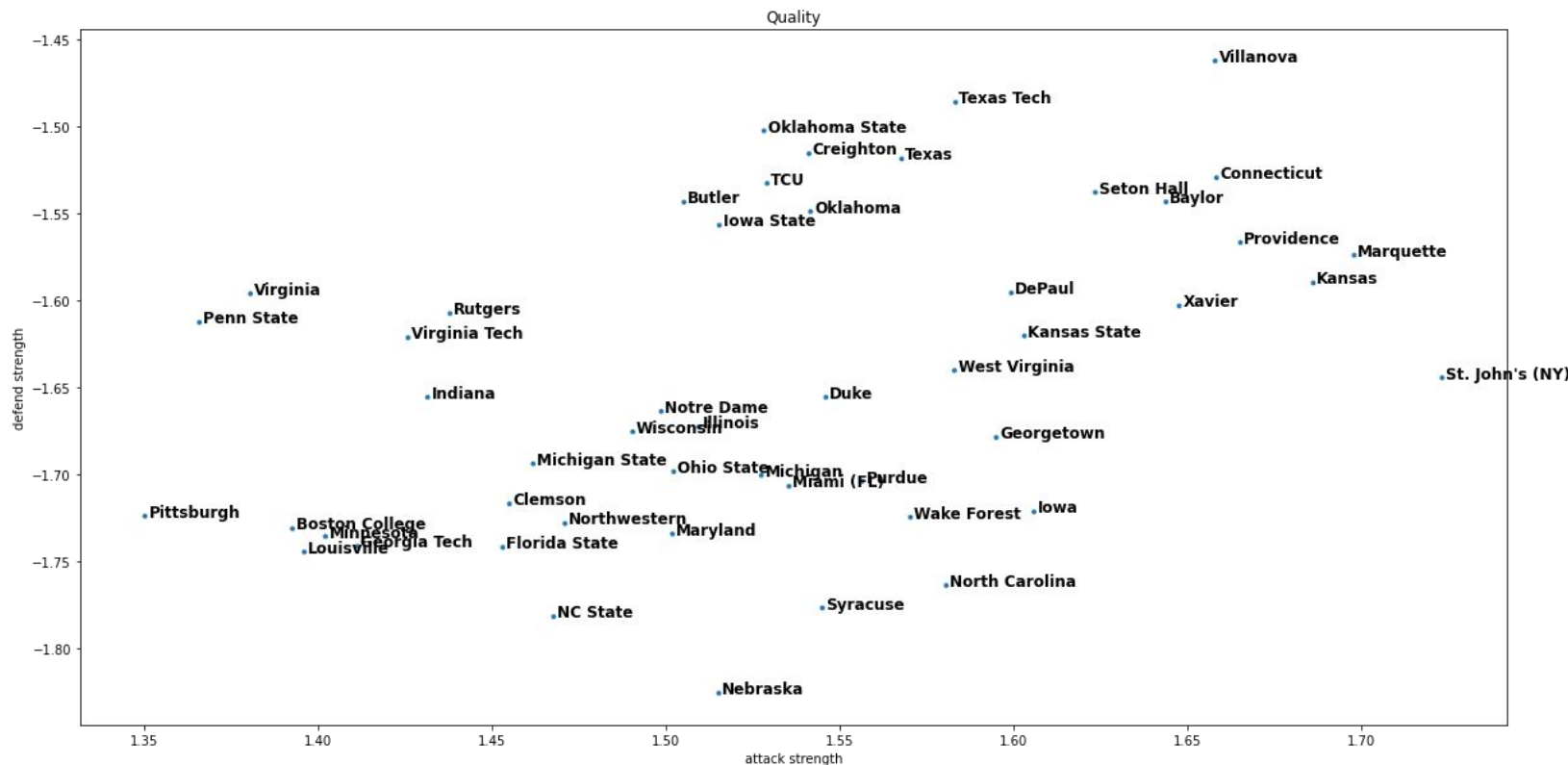


points



Data is from scores from regular
season games and are the
observed variables $s1$ and $s2$

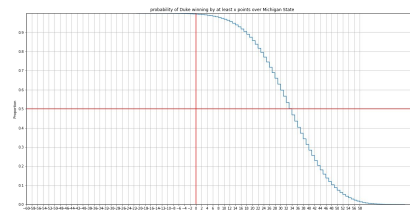
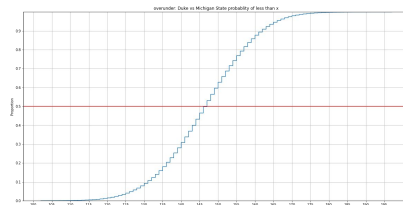
Output parameters seem reasonable



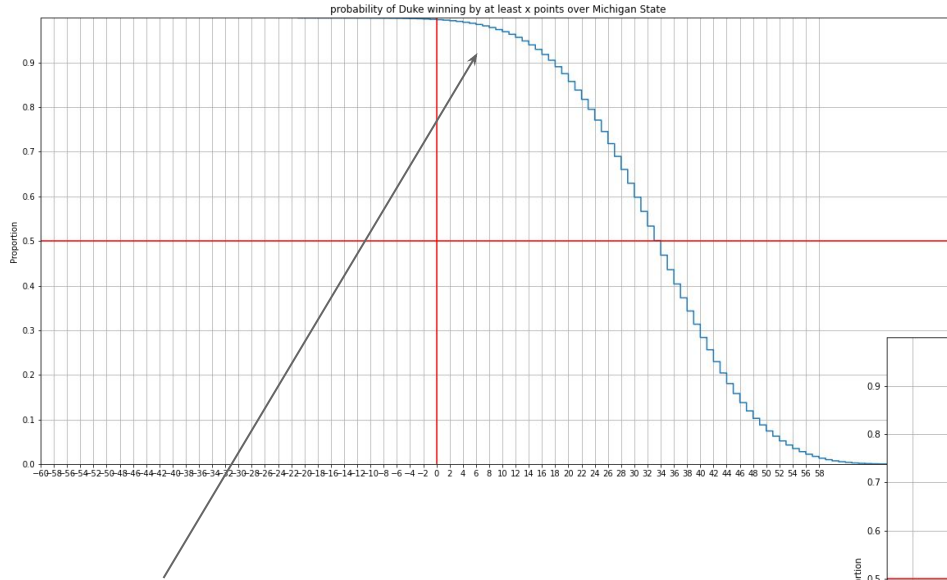
Agrees roughly with [external publications about offense and defense rankings!](#)

Using the generative model to make a decision

1. Select training data
2. Train model
3. Generate scores for matchup of interest (e.g., Michigan State vs Duke)
 - a. Sample parameters from their posterior distributions (e.g., `attack_msu`, `defense_duke`)
 - b. Sample score from resulting Poisson distribution (e.g., MSU's score)
 - c. Repeat (a) then (b) 1000 times
4. Generate scores for games
 - a. Pick, with replacement, a score from the 1000 for MSU and a score from the 1000 for Duke
 - b. Repeat 500 times
5. Compute cumulative distribution function for spread and sum-of-points

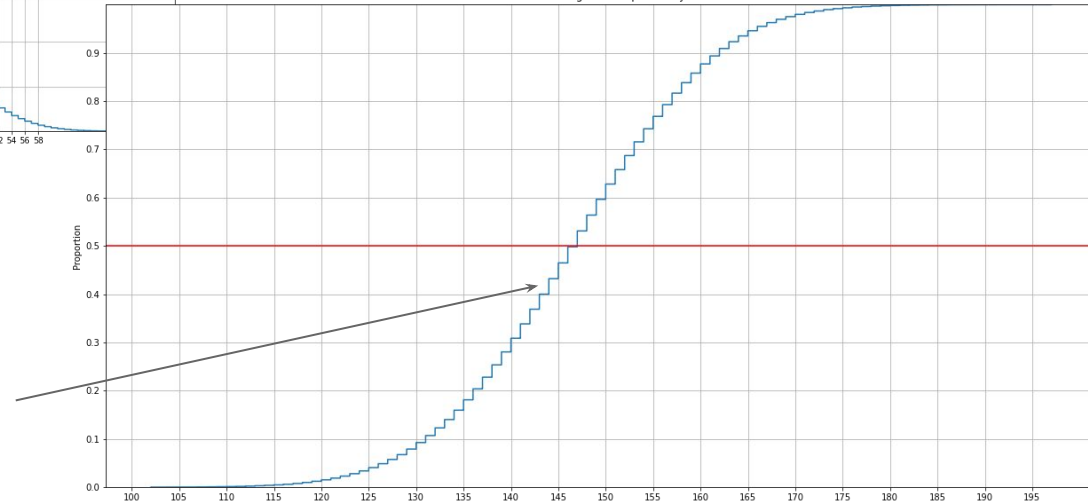


CDFs I used for betting



Spread of 6.5 points:
model thinks there is
a 98% chance Duke
will cover this

OverUnder of 144:
model thinks there is
a 43% the combined
points will be less



COLLEGE BASKETBALL

El Cortez
HOTEL & CASINO

Sunday, March 20, 2022

MEN'S TOURNAMENT - SECOND ROUND - GR

LOCAL TV TIME	BET #	TEAM	LINE	ML	FINAL	BET #	LINE	ML	FINAL
	815	MIAMI FL	144	+290		1815	68	+190	
4:45 PM	816	AUBURN	7 1/2	-350		1816	-4	-220	
	817	MICHIGAN ST	144	+235		1817	68	+175	
2:15 PM	818	DUKE	-6 1/2	-275		1818	-3 1/2	-200	

MEN'S TOURNAMENT - SECOND ROUND - M

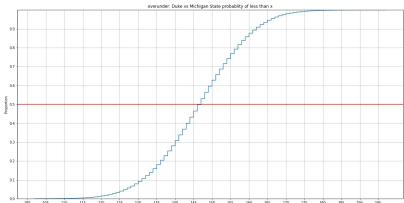
LOCAL TV TIME	BET #	TEAM	LINE	ML	FINAL	BET #	LINE	ML	FINAL
	819	TEXAS	133 1/2	+150		1819	63	+125	
5:40 PM	820	PURDUE	-3 1/2	-170		1820	-2	-145	

overunder: Duke vs Michigan State probability of less than x

The process in real life

1. Get up at 8am; drink Gatorade; drink coffee
2. Start with 9am games, looking up conferences of the teams
3. Filter data for those conferences plus a few others
4. Train a model; if it crashes, subtract out a conference and retry
5. Generate scores from the model for each team in matchup (1000x)
6. Sample with replacement from (step 5) outputs to get sample games
7. Record if the p50 prediction is different enough from line for OverUnder or Spread
8. Repeat for other games and push graphs to GitHub
9. Check bets with college roommate, Alec, who always was better with numbers
10. Rush down to the sportsbook to place bets; try to get lucky counter worker (the guy from Boston); tip him for good luck and ask for drink tickets
11. Order drink using a drink ticket and tip for good luck
12. Settle in for a long day

Action shots of process



COLLEGE BASKETBALL
Saturday, March 18, 2022

MEN'S TOURNAMENT - SECOND ROUND - BUFFALO, NY

TIME	HOME	AWAY	SCORE	TIME	HOME	AWAY	SCORE
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68

MEN'S TOURNAMENT - SECOND ROUND - INDIANAPOLIS, IN

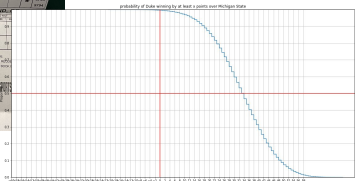
TIME	HOME	AWAY	SCORE	TIME	HOME	AWAY	SCORE
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68

MEN'S TOURNAMENT - SECOND ROUND - FORT WORTH, TX

TIME	HOME	AWAY	SCORE	TIME	HOME	AWAY	SCORE
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68

MEN'S TOURNAMENT - SECOND ROUND - PORTLAND, OR

TIME	HOME	AWAY	SCORE	TIME	HOME	AWAY	SCORE
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68
12:00	UTAH	UTAH STATE	75-68	12:00	UTAH	UTAH STATE	75-68



“Side-by-side” implementation: PPL model def

```
def model(home_id, away_id, score1_obs=None, score2_obs=None):
    # hyperpriors
    alpha = pyro.sample("alpha", dist.Normal(0.0, 1.0))
    sd_att = pyro.sample("sd_att", dist.TransformedDistribution(dist.StudentT(3.0, 0.0, 4), FoldedTransform()))
    sd_def = pyro.sample("sd_def", dist.TransformedDistribution(dist.StudentT(3.0, 0.0, 4), FoldedTransform()))

    # team-specific model parameters
    nt = len(list(set(np.unique(home_id).tolist()).union(set(np.unique(away_id).tolist()))))
    with pyro.plate("plate_teams", nt):
        attack = pyro.sample("attack", dist.Normal(0, sd_att))
        defend = pyro.sample("defend", dist.Normal(0, sd_def))

    # likelihood
    theta1 = torch.exp(alpha + attack[home_id] - defend[away_id])
    theta2 = torch.exp(alpha + attack[away_id] - defend[home_id])

    with pyro.plate("data", len(home_id)):
        pyro.sample("s1", dist.Poisson(theta1), obs=score1_obs)
        pyro.sample("s2", dist.Poisson(theta2), obs=score2_obs)
```

Code maps to the math but doesn't look like it exactly; inference is separate

with pyro.plate idiom implements conditionally independent similarly distributed plate concept

"Side-by-side" implementation: non-PPL model

```
def log_likelihood(  
    goals_home_observed, goals_away_observed, #will serve as k  
    home_attack, home_defence, away_attack, away_defence, home_advantage,):  
    #these will serve as mu  
    goal_expectation_home = np.exp(home_attack + away_defence + home_advantage)  
    goal_expectation_away = np.exp(away_attack + home_defence)  
  
    if goal_expectation_home < 0 or goal_expectation_away < 0:  
        return 10000  
    home_llk = poisson.pmf(goals_home_observed, goal_expectation_home)  
    away_llk = poisson.pmf(goals_away_observed, goal_expectation_away)  
  
    #probability of both teams' scores  
    log_llk = np.log(home_llk) + np.log(away_llk)  
    return -log_llk
```

*Hard to keep model
definition in one place*

*Easy for inference to
seeps into the model
definition*

This is great code from [Martin Eastwood](#). To avoid code duplication wrt the model definition and to isolated it from inference, would require building something PPL-like!

```
def predict(home_team, away_team, params, max_goals=10):  
    home_attack = params["attack_" + home_team]  
    home_defence = params["defence_" + home_team]  
    away_attack = params["attack_" + away_team]  
    away_defence = params["defence_" + away_team]  
    home_advantage = params["home_adv"]  
  
    home_goal_expectation = np.exp(home_attack + away_defence + home_advantage)  
    away_goal_expectation = np.exp(away_attack + home_defence)  
    home_probs = poisson.pmf(list(range(max_goals + 1)), home_goal_expectation)  
    away_probs = poisson.pmf(range(max_goals + 1), away_goal_expectation)  
  
    probability_matrix = np.outer(home_probs, away_probs)  
  
    return probability_matrix
```

Pyro inference

```
def guide(home_id, away_id, score1_obs=None, score2_obs=None):
    mu_locs = pyro.param("mu_loc", torch.tensor(0.0).expand(3))
    mu_scales = pyro.param("mu_scale", torch.tensor(0.1).expand(3),
                           constraint=constraints.positive)

    pyro.sample("alpha", dist.Normal(mu_locs[0], mu_scales[0]))
    pyro.sample("sd_att", dist.LogNormal(mu_locs[1], mu_scales[1]))
    pyro.sample("sd_def", dist.LogNormal(mu_locs[2], mu_scales[2]))

    nt = len(list(set(np.unique(home_id).tolist())
                  .union(set(np.unique(away_id).tolist()))))
    mu_team_locs = pyro.param("mu_team_loc",
                              torch.tensor(0.0).expand(2, nt))
    mu_team_scales = pyro.param("mu_team_scale",
                                torch.tensor(0.1).expand(2, nt),
                                constraint=constraints.positive)

    with pyro.plate("plate_teams", nt):
        pyro.sample("attack", dist.Normal(mu_team_locs[0],
                                          mu_team_scales[0]))
        pyro.sample("defend", dist.Normal(mu_team_locs[1],
                                          mu_team_scales[1]))
```

The variational inference gave reasonable results after 1000 iterations and took less than 30 seconds to run. The non-PPL optimization routine didn't finish for sets larger than 12 teams. Pyro's inference engines do seem to be the big differentiator to other PPLs such as Stan.

This code comes verbatim from Theo Rashid. I would have really struggled to write it myself.

```
def run_inference(model, guide, home_id, away_id,
                  score1, score2, args):
    gamma = 0.01#final learning rate gamma*initial_lr
    lrd = gamma ** (1 / args.num_iterations)
    svi = SVI(
        model=model,
        guide=guide,
        optim=ClippedAdam({"lr": args.learning_rate, "lrd": lrd}),
        loss=Trace_ELBO(num_particles=args.num_particles),
    )
    pyro.clear_param_store() # clear global parameter cache
    pyro.set_rng_seed(args.rng_seed)
    advi_loss = []
    for j in range(args.num_iterations):
        # calculate the loss and take a gradient step
        loss = svi.step(
            home_id=home_id,
            away_id=away_id,
            score1_obs=score1.float(),
            score2_obs=score2.float(),
        )
        advi_loss.append(loss)

    fit = Predictive(model=model, guide=guide, num_samples=2000)(
        home_id=home_id, away_id=away_id
    )
```

Non-PPL Inference

[Martin Eastwood](#) uses `scipy.optimize.minimize` to fit a simpler poisson regression model; set up is like basic optimization task.

```
def fit_poisson_model():
    teams = np.sort(np.unique(np.concatenate([big10df["HomeTeam"],
big10df["AwayTeam"]]))))
    n_teams = len(teams)

    params = np.concatenate(
        (np.random.uniform(0.5, 1.5, (n_teams)), #attack strength
         np.random.uniform(0, -1, (n_teams)), #defence strength
         [0.25], # home advantage)
    )

    def _fit(params, big10df, teams):
        attack_params = dict(zip(teams, params[:n_teams]))
        defence_params = dict(zip(teams,
                                params[n_teams : (2*n_teams)]))
        home_advantage = params[-1]
        llk = list()
        for idx, row in big10df.iterrows():
            tmp = log_likelihood(
                row["FTHG"], row["FTAG"],
                attack_params[row["HomeTeam"]],
                defence_params[row["HomeTeam"]],
                attack_params[row["AwayTeam"]],
                defence_params[row["AwayTeam"]],
                home_advantage,
            )
            llk.append(tmp)
        return np.sum(llk)
```

```
options = {
    "maxiter": 100,
    "disp": False,
}
constraints = [{"type": "eq", "fun":
    lambda x: sum(x[:n_teams]) - n_teams}]

res = minimize(
    _fit, params,
    args=(big10df, teams),
    constraints=constraints,
    options=options,
)
model_params = dict(
    zip(
        ["attack_" + team for team in teams]
        + ["defence_" + team for team in teams]
        + ["home_adv"],
        res["x"],
    )
)

return model_params
```

What it was like getting the Pyro to work

Input data was double, internal weight vector was float

```
pytorch RuntimeError: Expected object of scalar type Double but got scalar type Float
```

*You have to understand
pytorch tensors, which is
analogous to needing to know
numpy data structures to use
Scikit Learn.*

*Brutal error messages: this is a hard limit on the count of observation
variables or team variables (I think)*

```
ValueError: Error while computing log_prob at site 's1':  
Expected value argument (Tensor of shape (1798,)) to be within the  
support (IntegerGreaterThan(lower_bound=0)) of the distribution  
Poisson(rate: torch.Size([1798])), but found invalid values:  
tensor([73., 70., 92., ..., 66., 81., 97.])  
Trace Shapes:  
  Param Sites:
```

What it was like getting the Pyro to work

The prediction set has to have all the teams in it or it wouldn't run : mis-match in matrix dimensions...

I still don't understand how the prediction function accesses the trained parameters...

I wanted to train a home court advantage feature and a neutral court feature but could not get them to work.

Using someone else's code always has pros and cons:

- *you get a jumpstart on a medium complexity solution,*
- *but once you hit a snag, you have to put in time to deepen your understanding of the underlying system.*
- *But at least you are focusing your learning on stuff that matters.*
- *A good tutorial is worth its weight in gold*

Revisiting Marc's desiderata for PPLs

- Model specification should be
 -  Separate from inference methods
 -  In one place in the code
 -  Easy to verify that code implements math
 -  Code has direct mapping to math
 -  Visualization tools are provided
- Inference and generation should have
 -  Standardized interface making it easy to switch methods
 -  Details hidden for things like numerical underflow
 -  Tools for monitoring of progress, fit evaluation, & debugging tools
 -  Efficient implementation
- Documentation
 -  Meaningful error messages
 -   Great tutorial
 -  Cookbook of example models+inference (like the galleries for Matplotlib, Seaborn, ggPlot, etc.)

What about the drink tix, you ask?

My brother's guaranteed way to win in Vegas!

\$10 bet (expected value = $0.5 \cdot 0 + 0.5 \cdot 19 = \9.5)

\$2 tip to sportsbook worker and ask for drink tix

Receive 2 drink tix each worth \$5

Tip cocktail waitress/waiter \$2

ROI Investment
 $(9.5 + 10) - (10 + 2 + 2) = \$5.5 !$ (in value)

This only works on Fremont Street; don't try it on the Strip



Backup slide 1: the pros and cons of generative models

Consider a random forest model that predicts the spread directly from features of the two teams, e.g., mean points scored, wins against ranked teams, etc.

- What is the additional utility of the generative model?
 - Sum of scores can also be calculated
 - How often will MSU score more than 100 points
- What is the cost of that utility?
 - Optimizer might sacrifice spread accuracy to get point totals correct also

Going all in on generative model: how about generating each incremental scoring event, i.e., generate each basket, [random walk model](#); next year!!!

Backup slide 2: why do I think the model worked at all

1. How is the spread set: Vegas wants half the bets on one side and half on the other.
 - so you are betting against other people, not against Vegas
2. Regression to the mean: human bettors do not account for this and think that an outstanding performance the week before will be repeated but, on average, this is not what happens. Instead, teams tend to perform like they did during the regular season
 - therefore: the regular season really is predictive of tournament play
3. I dropped the home court advantage which seemed to be a reasonable thing to do
4. Only used major conferences so accounting for strength of schedule was less important than if I had put in all the small weaker conferences

Backup slide 3: problems and next steps

1. Spread is unreasonably big sometimes
 - a. [This](#) “random walk” paper suggests incorporating an effect where the team in the lead slows down and the team behind speeds up in their respective scoring rates
 - b. I am not sure what this would do for the over under; perhaps little especially if the rate changes were offsetting
2. The overunder was biased high it seemed;
 - a. Need to check this.
 - b. One could simple look at the distribution of scores involving only big conferences in the tournament to the distribution during the regular season conference games; then adjust the Poisson accordingly